

Smart-Home IoT Anomaly Detection

at the Gateway, with a Pre-trained Mamba Detector

Madhav Dogra • Project Proposal

SNAPSHOT

A detector that sits at the home gateway and inspects every flow going to or from a smart-home device. It catches **known attacks** via a supervised classifier, and **zero-day attacks** via a one-class anomaly scorer. The whole thing rides on a single **Mamba** encoder pre-trained on benign traffic; both heads share it. Two operating modes: a low-power **default** that runs continuously, and a stronger **admin-switched** mode for when the situation calls for it. New attack types are absorbed by re-training only the lightweight classifier head — not the encoder. Designed to run at <10 ms p95 on Raspberry-Pi-class hardware.

1 The setting

The target environment is the modern smart home. The device population we care about includes **IP cameras, motion / environmental sensors, smart bulbs, voice assistants, thermostats, video doorbells, smart plugs, and the home gateway itself**. CICIoT2023, our primary training dataset, is dominated by exactly this device class: 105 real consumer IoT devices, 33 attack types, 7 attack categories.

1.1 Why these devices are the soft layer

They ship with default credentials, run firmware that is rarely updated, lack the compute to host any agent or endpoint security tool, and are often physically accessible. The Mirai botnet — built almost entirely from compromised IP cameras and home routers — is the canonical demonstration that this surface is exploitable at scale.

1.2 Why the gateway is the right vantage point

Putting software *on* the devices is not possible. Watching them on the network is. The home gateway is the single point through which every device flow must pass on its way to the internet. It is the only place where modern compute can sit without instrumenting the device fleet. The trade-off: gateway hardware is small (Raspberry-Pi-class, single-digit watts), so the detector must be lightweight. This single constraint shapes every architectural decision that follows.

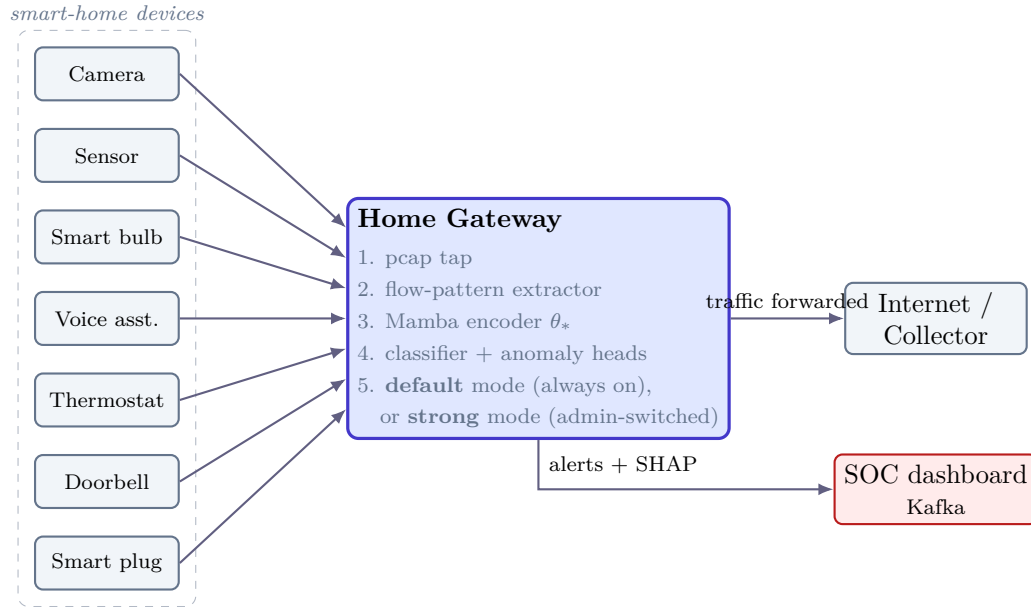


Figure 1: *

Figure 1 — The detector lives inside the home gateway. Device traffic continues to the internet unchanged; alerts and SHAP attributions are sent out-of-band to a SOC dashboard.

2 The approach in one diagram

Conventional IoT IDS collapses each flow into a vector of ~ 80 summary statistics (mean packet size, IAT standard deviation, flag counts, ...) before showing it to the model. This is convenient but *permutation-invariant* — you can shuffle the packets and the statistics don't change — which destroys the temporal structure that attack signatures live in.

KEY IDEA

Keep the order. A flow is the first $K \approx 32$ packets of the connection, kept as an ordered sequence $\mathbf{x} = (p_1, \dots, p_K)$. Each p_t is a small per-packet feature vector — size, direction, inter-arrival time, TCP flags, header fields. A sequence model (Mamba) can then learn what a normal sequence looks like, and what a malicious one looks like.

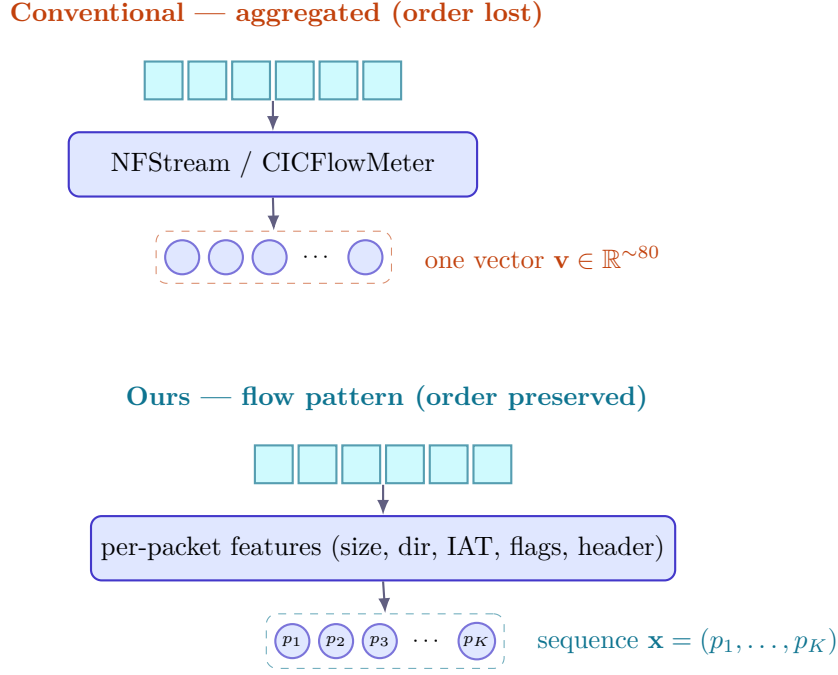


Figure 2: *

Figure 2 — Two ways to feed a flow to a model. Aggregation collapses everything into one vector and drops ordering; the flow-pattern keeps the packets in order so a sequence model can use the temporal structure.

3 The architecture

A single **Mamba encoder** processes the sequence $\mathbf{x}$ and produces a fixed-size embedding $\mathbf{z}$. Two heads then read $\mathbf{z}$, in parallel, for two purposes:

- **Classifier head** — a small MLP $\rightarrow$ softmax over the 7 known attack categories from CICIoT2023. Trained with focal loss to handle the heavy class imbalance.
- **Anomaly head** — a one-class deep SVDD scorer, trained *only on benign embeddings*. It does not know about any specific attack; it just measures distance from "normal."

KEY IDEA

The anomaly head doesn't read $\mathbf{z}$ directly. It reads a **projected** embedding $\mathbf{z}_{\text{ano}} = g_{\phi}(\mathbf{z})$ produced by a small learnable MLP (e.g. $\mathbb{R}^{128} \rightarrow \mathbb{R}^{32}$). One-class scorers like deep SVDD suffer from the curse of dimensionality — in high dimensions, distances concentrate and lose discriminative power. Projecting down to a smaller anomaly space sharpens the score. The projection is trained jointly with the SVDD objective.

3.1 Why Mamba and not a transformer

The reference architecture for pre-trained encrypted-traffic detectors is ET-BERT, a BERT-base-scale ($\sim 110\text{M}$ parameter) transformer. It works but doesn't fit on a Raspberry Pi. NetMamba (2024) showed that a Mamba state-space model achieves ET-BERT-class accuracy at substantially lower parameter and latency cost — because Mamba runs in linear time in sequence length and has much lower per-token overhead than attention. We adopt that design.

3.2 Why a shared encoder

Pre-training is the expensive part. Sharing the encoder between both heads means we pay it once. It also gives the anomaly head a representation of "normal" that has been learned from a large benign corpus — exactly what an anomaly scorer needs.

4 Two operating modes — default and strong

The gateway can run in one of two configurations. The choice is an **operator-level setting** flipped by the admin, not a per-flow decision. One mode runs continuously by default; the other is switched on when the situation calls for it.

- **Default mode — low power, always on.** Smaller Mamba configuration, smaller K (e.g., $K = 8$), *only the anomaly head* active, no SHAP. This is the baseline operating point: detection is on, the compute cost is small enough for continuous use on Raspberry-Pi-class hardware, and the system catches both known and novel attacks via the anomaly score alone.
- **Strong mode — admin-switched, full pipeline.** Full Mamba configuration, $K = 32$, *both* classifier and anomaly heads (the latter via the projection MLP), plus SHAP attribution on every alert. Activated by the admin when needed — elevated threat level, active incident investigation, high-value time window, or simply when the operator wants the maximum detection fidelity.

Switching is a configuration change applied to the gateway, not a per-flow decision and not an escalation. The same model artifacts — encoder θ_* , classifier head, projection, anomaly head — are used in both modes; what differs is which components are engaged and at what size. Both modes produce alerts that go to the SOC. Strong mode additionally produces per-packet SHAP attributions for every alert.

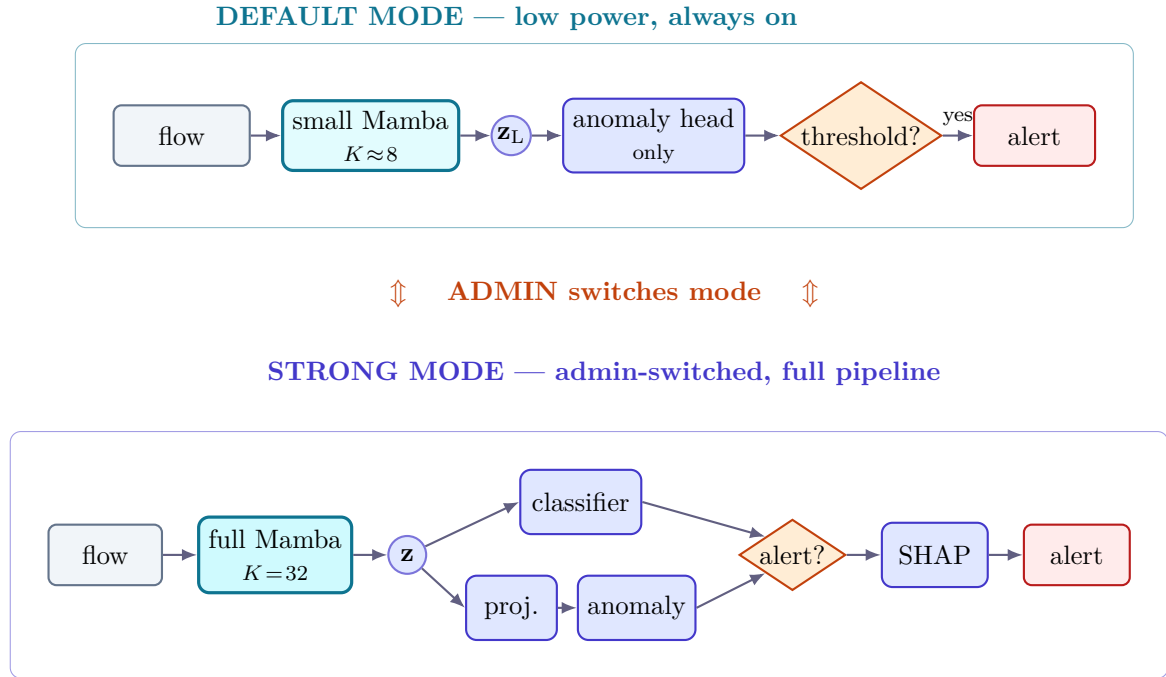


Figure 3: *

Figure 3 — The two operating modes. Default mode runs continuously and uses only the lightweight anomaly head on a small Mamba. Strong mode is switched on by the admin and engages the full pipeline: full Mamba, classifier head, projected anomaly head, and SHAP attributions. The admin chooses which mode is active; the gateway then operates entirely in that mode until the next switch.

5 Training — three stages, then a loop

The detector is built in three stages. Each stage produces an artifact the next stage uses. The encoder is built first (without labels), then specialised with labels, then the anomaly head is added on top.

5.1 Stage A — Pre-train the encoder, no labels

Take a large set of **benign** flows. Randomly mask 15% of packet positions in each sequence. Train the Mamba encoder to predict the masked packets from context. This is the SSL recipe pioneered by ET-BERT, applied to per-packet features. The output is an encoder θ_0 that has learned what normal traffic looks like.

5.2 Stage B — Fine-tune with labels

Attach a fresh classification head on top of θ_0 . Jointly fine-tune the encoder and the head on labelled CICIOT2023 attack data with focal loss. The output is the fine-tuned encoder θ_* and the trained classifier.

5.3 Stage C — Fit the anomaly head

Freeze θ_* . Push a held-out benign set through θ_* to get embeddings. Train the projection MLP and fit deep SVDD on the projected embeddings. Calibrate the threshold at the 99th percentile of benign scores (1% FPR target).

5.4 The loop — absorbing new attack types

When a previously unseen attack type appears in the wild, the system does *not* need full retraining. The flow is:

1. Anomaly head flags the novel attack on day zero (it just looks unlike normal).
2. SOC labels a sample of the flagged flows as a new attack class.
3. Extend the classifier softmax by one class and fine-tune **only the classifier head** on the new data, with a small replay buffer of old classes to prevent forgetting. The encoder θ_* stays frozen. Minutes, not days.
4. Optionally recalibrate the anomaly threshold on a fresh benign set.

Full encoder retraining is only triggered if the benign distribution itself shifts (new device types added to the network). This decoupling — "what's normal" lives in the expensive encoder, "what's malicious" lives in the cheap heads — is the operational payoff of the shared-encoder design.

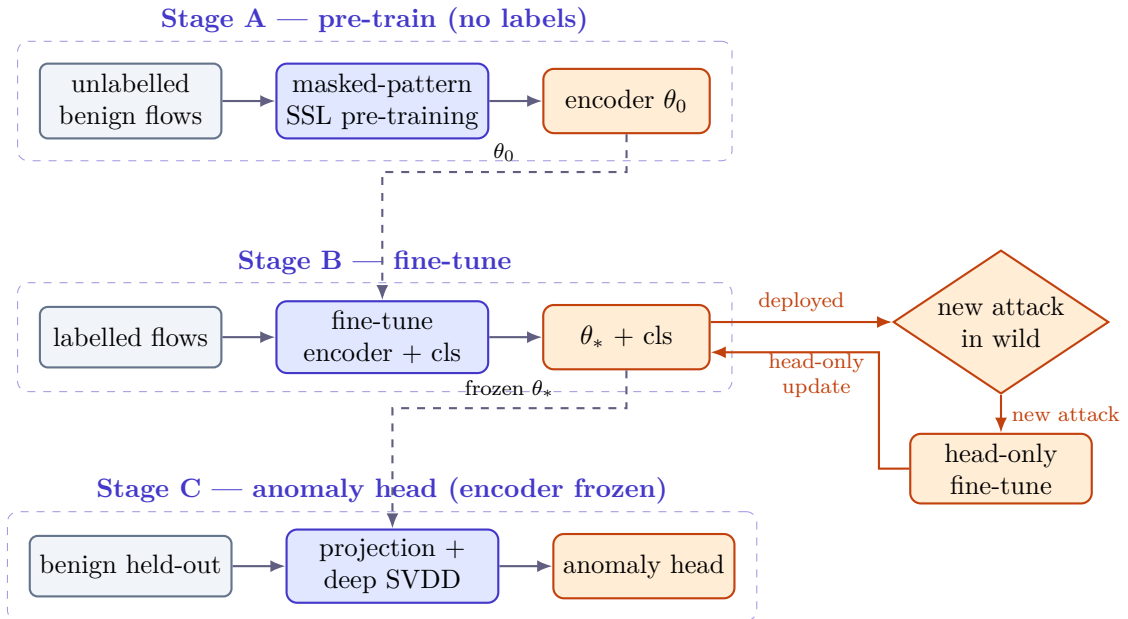


Figure 4: *

Figure 4 — The three training stages produce the deployed detector. The amber loop on the right is the progressive-learning path: when a new attack arrives, only the lightweight classifier head is re-trained — the expensive encoder stays put.

6 Evaluation

6.1 Datasets

CICIoT2023 (primary). **TON_IoT** for cross-dataset generalisation — trained model evaluated on a dataset it has never seen. **IoT-23** for a real-world malware check (Mirai, Torii, Okiru, Gafgyt).

6.2 Baselines — and what each one is for

- **ET-BERT** — the published-SOTA upper bound. Tells us how much accuracy we leave on the table by going to Mamba.
- **CNN-BiLSTM on aggregated statistics** — the representation control. Isolates the gain from the flow-pattern input.
- **XGBoost on aggregated statistics** — the *classical-floor challenge*. In IDS literature, gradient-boosted trees on hand-crafted features are routinely competitive with deep models. We keep XGBoost as an explicit baseline to answer the “do we need deep learning at all?” question. The deep detector only earns its complexity if it beats XGBoost on the metrics XGBoost cannot target: **zero-day detection** (XGBoost is supervised-only), **cross-dataset transfer** (the pre-trained encoder is the bet), and **adversarial robustness**. If XGBoost matches us in-distribution, we report that as a finding.
- **Isolation Forest** — the classical unsupervised floor against which the anomaly head is judged.

6.3 Metrics

Per-class precision / recall / F1. PR-AUC (the honest metric under heavy class imbalance). Inference latency p50 / p95 / p99 per flow. Model size in MB after int8 quantisation.

6.4 Adversarial study

Fast Gradient Sign Method (FGSM) on the per-packet sequence, perturbations clipped to valid feature ranges. Detection rate vs. perturbation magnitude ε , plotted for both heads. Secondary check: do SHAP attribution patterns on adversarial inputs differ from those on clean inputs? (They tend to — a useful fallback signal.)

6.5 Zero-day proxy

Leave-one-class-out: hide an attack class from the supervised head during Stage B, then measure how well the anomaly head detects it at test time. Repeat per class. This is our concrete answer to “how do you evaluate something you haven’t trained on.”

7 Plan, deliverables, risks

7.1 Twelve-week plan

Weeks 1–2 setup and dataset validation; week 3 flow-pattern extractor and classical baselines; weeks 4–5 Stage A pre-training; week 6 Stage B fine-tuning and bring up CNN-BiLSTM + ET-BERT comparison; week 7 Stage C anomaly head and leave-one-class-out; week 8 TON_IoT + IoT-23 evaluation; week 9 SHAP integration; week 10 FGSM adversarial study; week 11 int8 quantisation and gateway latency benchmark; week 12 stretch goal (if time), final report, presentation.

7.2 Deliverables

Reproducible code repository covering the full pipeline. Pre-trained Mamba encoder checkpoint θ_* released alongside the report. Benchmark table covering Mamba vs. ET-BERT vs. CNN-BiLSTM vs. XGBoost vs. Isolation Forest across CICIoT2023, TON_IoT, IoT-23. FGSM degradation curves. Latency / memory profile on gateway hardware. Written report.

7.3 Risks — and what I do about them

WATCH OUT

- **Pre-training compute blows budget.** Warm-start from a public Mamba checkpoint and adapt on benign IoT traffic rather than train from scratch.
- **Class imbalance hides minority-class failure.** Focal loss + always report per-class metrics.
- **Cross-dataset accuracy drop.** Measure honestly; this is the standard caveat in IDS literature, not a failure mode to hide.
- **Edge memory overrun.** Int8 quantise. Fall back to a smaller Mamba configuration. Reduce K as a last resort.
- **Byte-level adversarial sensitivity.** The FGSM study quantifies it explicitly. SHAP fingerprint as a secondary defence.

7.4 Stretch goals

- Swap Mamba $\rightarrow$ Mamba-2 (Structured State Space Duality) if latency headroom allows.
- Add an E-GraphSAGE branch over a per-window device-communication graph.
- Federated pre-training across multiple homes / sites.

8 Key references

- NetMamba, Wang et al. 2024 — arxiv.org/abs/2405.11449
- ET-BERT, Lin et al. WWW 2022 — arxiv.org/abs/2202.06335
- Mamba, Gu & Dao 2023 — arxiv.org/abs/2312.00752
- Mamba-2 / SSD, Dao & Gu 2024 — arxiv.org/abs/2405.21060
- S4D, Gu, Goel, Ré 2022 — arxiv.org/abs/2206.11893
- CICIOT2023, Neto et al. 2023 — mdpi.com/1424-8220/23/13/5941
- Deep SVDD, Ruff et al. ICML 2018 — the one-class anomaly scorer
- ET-SSL, Nature Sci. Rep. 2025 — contrastive SSL on encrypted traffic
- E-GraphSAGE, Lo et al. — arxiv.org/abs/2103.16329